



UNIVERSIDAD NACIONAL DE ASUNCIÓN

PROYECTO 1

ROBOT AUTO-EQUILIBRADO

Autores:

Victor Gabriel Curiel Gonzalez
Hernan Gabriel Espinola Fleitas
Ximena Lujan Quenhan Riveros

Profesor:

Ing. Luis Poisson
Ing. Francisco Gonzalez

Fecha: 21 de Junio 2024

Índice

1. Introducción.	2
2. Diseño del proyecto.	2
2.1. Componentes.	2
2.2. Diagrama de bloques.	6
2.3. Montaje y pruebas.	6
3. Código del programa.	8
4. Conclusión.	10

1. Introducción.

La robótica ha avanzado significativamente, permitiendo la creación de sistemas autónomos cada vez más complejos y eficientes. Un ejemplo notable de estos avances es el robot autoequilibrado. Estos robots son capaces de mantenerse en equilibrio sobre dos ruedas, utilizando principios de control y retroalimentación. Este informe se enfoca en la construcción y funcionamiento de un robot autoequilibrado que emplea un Arduino Uno, un giroscopio, un puente H y dos motores DC.

El objetivo principal de este proyecto es demostrar cómo un sistema simple puede utilizar sensores y actuadores para mantener el equilibrio dinámico. La importancia de este tipo de robots radica no solo en su aplicación práctica, como en robots de asistencia y juguetes inteligentes, sino también en su valor educativo. Al comprender el funcionamiento de un robot autoequilibrado, se pueden aprender conceptos fundamentales de electrónica, programación y teoría de control.

2. Diseño del proyecto.

2.1. Componentes.

Arduino 1: Arduino es una plataforma de creación de electrónica de código abierto, basada en hardware y software libre, flexible y fácil de utilizar para los creadores y desarrolladores. Esta plataforma permite crear diferentes tipos de microordenadores de una sola placa a los que se puede darles diferentes tipos de uso.

Arduino ofrece la plataforma Arduino IDE (Entorno de Desarrollo Integrado), que es un entorno de programación con el que cualquiera puede crear aplicaciones para las placas Arduino. Es una placa con todos los elementos necesarios para conectar periféricos a las entradas y salidas de un microcontrolador, y que puede ser programada tanto en Windows como macOS y GNU/Linux.

El Arduino es una placa basada en un microcontrolador ATMEL. Los microcontroladores son circuitos integrados en los que se pueden grabar instrucciones, las cuales las escribes con el lenguaje de programación que puedes utilizar en el entorno Arduino IDE. Estas instrucciones permiten crear programas que interactúan con los circuitos de la placa.

El microcontrolador de Arduino posee lo que se llama una interfaz de entrada, que es una conexión en la que podemos conectar en la placa diferentes tipos de periféricos. La información de estos periféricos se trasladará al microcontrolador, el cual se encargará de procesar los datos que le lleguen a través de ellos. También cuenta con una interfaz de salida, que es la que se encarga de llevar la información que se ha procesado en el Arduino a otros periféricos. Estos periféricos pueden ser pantallas o altavoces en los que reproducir los datos procesados, pero también pueden ser otras placas o controladores.



Figura 1: Arduino 1

Driver L298N: El L298N es un controlador de motores de doble puente H que permite controlar la velocidad y la dirección de dos motores DC de manera independiente. Es un componente esencial en proyectos de robótica que requieren movimiento controlado.

Principio de Funcionamiento:

- **Puente H:** Es un circuito que permite que un motor DC gire en ambas direcciones. Consiste en cuatro interruptores (transistores o MOSFETs) dispuestos en una configuración de puente. Al activar diferentes combinaciones de estos interruptores, se puede cambiar la dirección del flujo de corriente a través del motor, controlando así su dirección de rotación.
- **Control de velocidad:** Utilizando la técnica de modulación por ancho de pulso (PWM), es posible controlar la velocidad del motor. El L298N permite aplicar una señal PWM a sus entradas de habilitación para regular la velocidad de los motores conectados.

Características y Beneficios:

- **Capacidad de corriente:** El L298N puede manejar hasta 2 amperios por canal, lo que lo hace adecuado para controlar motores de tamaño pequeño a mediano.
- **Protección térmica:** Incluye protección contra sobrecalentamiento, asegurando la durabilidad del dispositivo en condiciones de alta carga.
- **Facilidad de uso:** Con una interfaz sencilla de pines de entrada y salida, el L298N se puede integrar fácilmente con microcontroladores y otros circuitos de control.

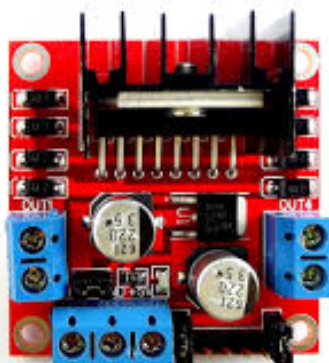


Figura 2: Modulo L298N motor driver

Giroscopio MPU6050: El MPU-6050 es un módulo sensor que combina un giroscopio de 3 ejes y un acelerómetro de 3 ejes en un solo chip. Este sensor es ampliamente utilizado en aplicaciones robóticas y de control debido a su capacidad para proporcionar datos precisos sobre la orientación y el movimiento del dispositivo.

Principio de Funcionamiento:

- **Acelerómetro:** Mide la aceleración lineal en las direcciones X, Y y Z. Esto permite detectar el movimiento y la inclinación del dispositivo. Los acelerómetros funcionan basándose en el principio de la piezoelectricidad, donde un material piezoeléctrico genera una carga eléctrica en respuesta a una fuerza aplicada.
- **Giroscopio:** Mide la velocidad angular alrededor de los ejes X, Y y Z. Utiliza el efecto Coriolis, donde una masa vibrante dentro del sensor experimenta una fuerza proporcional a la velocidad de rotación, permitiendo así la medición de la rotación.

Características y Beneficios:

- **Alta precisión:** El MPU-6050 proporciona mediciones precisas de aceleración y velocidad angular, lo que es crucial para aplicaciones de control de movimiento y estabilización.
- **Fusión de sensores:** El módulo incluye un procesador de movimiento digital (DMP) que realiza la fusión de datos del giroscopio y el acelerómetro, proporcionando datos de orientación (quaternion) que pueden ser utilizados directamente por el controlador del robot.
- **Interfaz I2C:** Facilita la comunicación con microcontroladores como el Arduino Uno, permitiendo una integración sencilla en proyectos de robótica.



Figura 3: Sensor MPU6050

Motores DC:

Los motores de corriente continua (DC) son dispositivos electromecánicos que convierten la energía eléctrica en energía mecánica. Son ampliamente utilizados en aplicaciones de robótica debido a su simplicidad, eficiencia y capacidad de control preciso.

Principio de Funcionamiento Conmutación: Los motores DC utilizan un conmutador mecánico para invertir la dirección de la corriente en las bobinas del rotor a medida que este gira. Esto crea un par que mantiene el rotor en movimiento. **Campo magnético:** Un motor DC típico consiste en un rotor (parte móvil) y un estator (parte fija). Al aplicar una corriente eléctrica a las bobinas del rotor, se genera un campo magnético que interactúa con el campo del estator, produciendo movimiento. **Características y Beneficios** **Control de velocidad:** La velocidad de un motor DC se puede controlar fácilmente variando el voltaje de entrada o utilizando técnicas de PWM. Esto permite un control preciso del movimiento en aplicaciones robóticas. **Par y velocidad ajustables:** Los motores DC están disponibles en una amplia gama de especificaciones de par y velocidad, permitiendo seleccionar el motor adecuado para cada aplicación específica. **Simplicidad y eficiencia:** Los motores DC son relativamente simples en diseño y construcción, lo que los hace eficientes y confiables para una variedad de aplicaciones.



Figura 4: DC TT Gear motor

2.2. Diagrama de bloques.

Se presenta el diagrama de bloques utilizado para realizar el proyecto.



Figura 5: Diagrama de bloques del proyecto

2.3. Montaje y pruebas.

A continuacion, se muestran las fotos del proceso de montaje del proyecto.

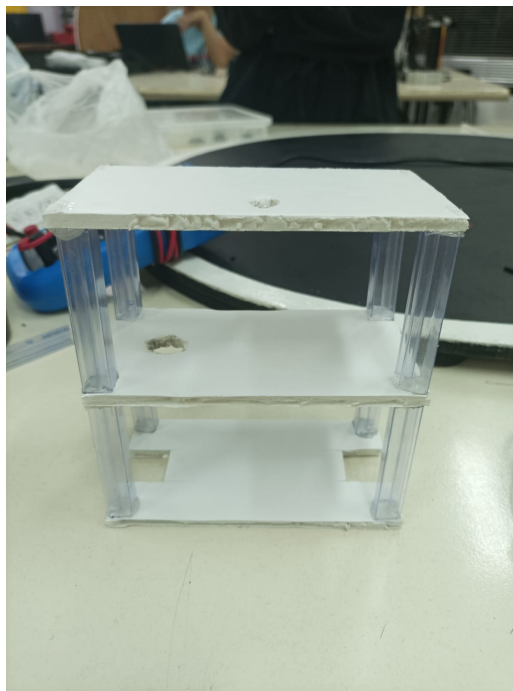


Figura 6: Estructura base

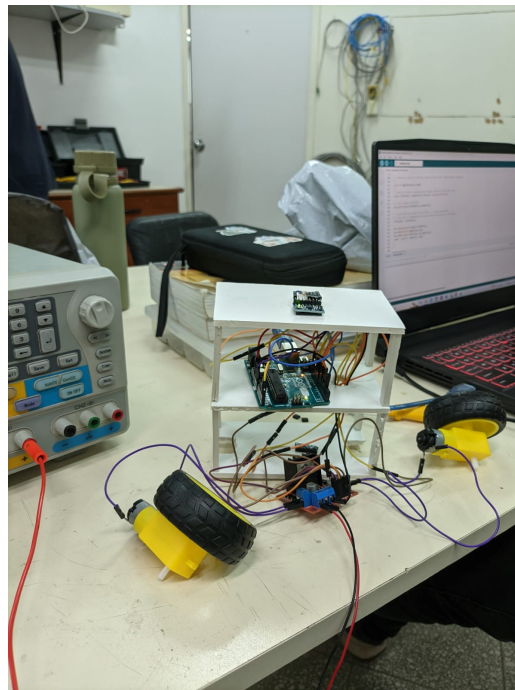


Figura 7: Componentes montados

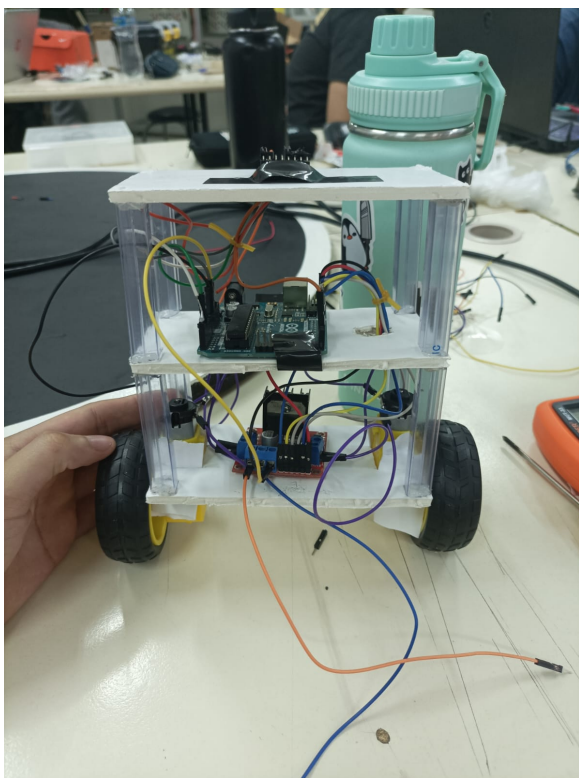


Figura 8: Primeras pruebas

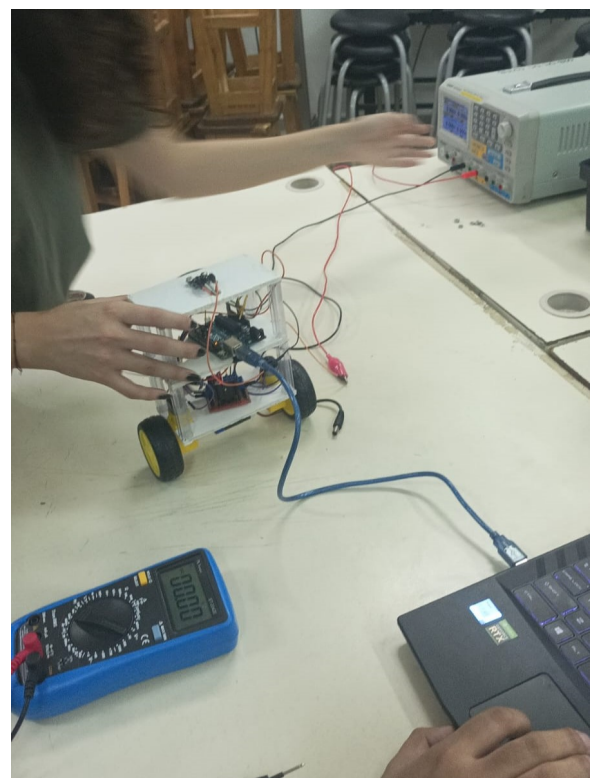


Figura 9: Pruebas finales

3. Código del programa.

```

1
2 #include <PID_v1.h>
3 #include <LMotorController.h>
4 #include "I2Cdev.h"
5 #include "MPU6050_6Axis_MotionApps20.h"
6
7 #if I2CDEV_IMPLEMENTATION == I2CDEV_ARDUINO_WIRE
8 #include "Wire.h"
9 #endif
10
11 #define MIN_ABS_SPEED 15
12
13 MPU6050 mpu;
14
15 bool dmpReady = false;
16 uint8_t mpuIntStatus;
17 uint8_t devStatus;
18 uint16_t packetSize;
19 uint16_t fifoCount;
20 uint8_t fifoBuffer[64];
21
22 Quaternion q;
23 VectorFloat gravity;
24 float ypr[3];
25
26 double originalSetpoint = 173;
27 double setpoint = originalSetpoint;
28 double movingAngleOffset = 0.1;
29 double input, output;
30
31 double Kp = 60;
32 double Kd = 1.4 ;
33 double Ki = 70;
34 PID pid(&input, &output, &setpoint, Kp, Ki, Kd, DIRECT);
35
36 double motorSpeedFactorLeft = 0.6;
37 double motorSpeedFactorRight = 0.6;
38 int ENA = 5;
39 int IN1 = 6;
40 int IN2 = 7;
41 int IN3 = 9;
42 int IN4 = 8;
43 int ENB = 10;
44 LMotorController motorController(ENA, IN1, IN2, ENB, IN3, IN4, motorSpeedFactorLeft,
    motorSpeedFactorRight);
45
46 volatile bool mpuInterrupt = false;
47 void dmpDataReady()
48 {
49     mpuInterrupt = true;
50 }
51
52 void setup()
53 {
54     #if I2CDEV_IMPLEMENTATION == I2CDEV_ARDUINO_WIRE
55         Wire.begin();
56         TWBR = 24;

```

```

57 #elif I2CDEV_IMPLEMENTATION == I2CDEV_BUILTIN_FASTWIRE
58     Fastwire::setup(400, true);
59 #endif
60
61     mpu.initialize();
62
63     devStatus = mpu.dmpInitialize();
64
65     mpu.setXGyroOffset(220);
66     mpu.setYGyroOffset(76);
67     mpu.setZGyroOffset(-85);
68     mpu.setZAccelOffset(1788);
69
70     if (devStatus == 0)
71     {
72         mpu.setDMPEnabled(true);
73         attachInterrupt(0, dmpDataReady, RISING);
74         mpuIntStatus = mpu.getIntStatus();
75         dmpReady = true;
76         packetSize = mpu.dmpGetFIFOPacketSize();
77         pid.SetMode(AUTOMATIC);
78         pid.SetSampleTime(10);
79         pid.SetOutputLimits(-255, 255);
80     }
81     else
82     {
83         Serial.print(F("DMP Initialization failed (code "));
84         Serial.print(devStatus);
85         Serial.println(F(")"));
86     }
87 }
88
89 void loop()
90 {
91     if (!dmpReady) return;
92
93     while (!mpuInterrupt && fifoCount < packetSize)
94     {
95         pid.Compute();
96         motorController.move(output, MIN_ABS_SPEED);
97     }
98
99     mpuInterrupt = false;
100     mpuIntStatus = mpu.getIntStatus();
101     fifoCount = mpu.getFIFOCount();
102
103     if ((mpuIntStatus & 0x10) || fifoCount == 1024)
104     {
105         mpu.resetFIFO();
106         Serial.println(F("FIFO overflow!"));
107     }
108     else if (mpuIntStatus & 0x02)
109     {
110         while (fifoCount < packetSize) fifoCount = mpu.getFIFOCount();
111         mpu.getFIFOBytes(fifoBuffer, packetSize);
112         fifoCount -= packetSize;
113
114         mpu.dmpGetQuaternion(&q, fifoBuffer);
115         mpu.dmpGetGravity(&gravity, &q);
116         mpu.dmpGetYawPitchRoll(ypr, &q, &gravity);

```

```
117     input = ypr[1] * 180/M_PI + 180;  
118 }  
119 }
```

4. Conclusión.

En este proyecto, hemos desarrollado y probado un robot autoequilibrado que integra diversos componentes electrónicos y mecánicos, utilizando el Arduino Uno como cerebro del sistema. A través de este proceso, hemos explorado conceptos clave en robótica y control, destacando la importancia de la sinergia entre sensores y actuadores.

El uso del MPU6050 como giroscopio y acelerómetro ha permitido captar la orientación y el movimiento del robot, datos que fueron fundamentales para el equilibrio del sistema. El controlador de motores L298N, junto con los motores DC, proporcionó la capacidad de ajustar la velocidad y dirección de los motores, permitiendo al robot mantenerse en pie y reaccionar a las perturbaciones externas.

La implementación del código basado en el algoritmo PID permitió una respuesta eficiente y estable del sistema de control, ajustando continuamente la velocidad de los motores para mantener el equilibrio.

El robot autoequilibrado ofrece un ejemplo práctico de cómo los sistemas de control pueden aplicarse en diversas áreas, desde la robótica de asistencia hasta la enseñanza de conceptos avanzados en mecatrónica. Las habilidades adquiridas en este proyecto, como el diseño de circuitos, la programación y la integración de sistemas, son aplicables a una amplia gama de aplicaciones en ingeniería y tecnología.